

Descrição

Descreva a diferença entre uma árvore, uma árvore binária e uma árvore de busca binária.

Resposta Discursiva**Nota Máxima**

1,80

Nota

0,00

luno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno/18122620232?idDisciplinaOferta=136731&token=PTv2ryr9MQ8%3D

23, 20:12

Portal do Aluno

Comentário da Resposta**Resposta Resumida:**

Árvore: estrutura com nós e filhos.

Árvore binária: cada nó tem até 2 filhos.

Árvore de busca binária: árvore binária onde menores ficam à esquerda e maiores à direita.

Descrição

Qual a diferença de uma árvore binária de busca para uma simples?

Resposta Discursiva

Toda BST, árvore binária de busca, é uma árvore binária, mas nem toda árvore binária é uma BST. A diferença entre elas está no fato das árvores binárias comuns apenas definirem uma estrutura em que um nó qualquer pode possuir no máximo dois nós filhos, um esquerdo e outro direito. Enquanto, a BST, além de implementar a mesma estrutura, também define uma ordenação para seus nós de forma que os nós à esquerda de uma raiz sempre terão um valor menor que ela, enquanto os nós à sua direita sempre terão um valor maior que o dela. Assim, graças a essa ordenação, a BST possibilita realizar a busca binária em sua estrutura.

Nota Máxima

1,80

Nota

1,80

Comentário da Resposta**Resposta Resumida:**

Árvore binária simples: só limita a 2 filhos por nó.

Árvore binária de busca (BST): além disso, tem ordem: menores à esquerda e maiores à direita, facilitando a busca.

Descrição

Qual a diferença de uma árvore binária de busca para uma AVL?

Resposta Discursiva

Uma árvore binária de busca é uma árvore que cada nodo possui no máximo dois filhos e dado um nodo qualquer dessa árvore, todos os elementos à sua esquerda devem ser menores que esse nodo assim como todos os elementos à sua direita devem ser maiores que esse nodo. Uma árvore AVL é uma árvore binária de busca balanceada, que recebe um fator balanceador. Esse fator representa o maior nível da subárvore esquerda menos o maior nível da subárvore direita ($FB = NE - ND$). Para ser considerada balanceada uma árvore AVL deve ter um fator balanceador +1, 0 ou -1. Caso esteja desbalanceada existem estratégias para fazer seu balanceamento.

Nota Máxima

1,80

Nota

1,80

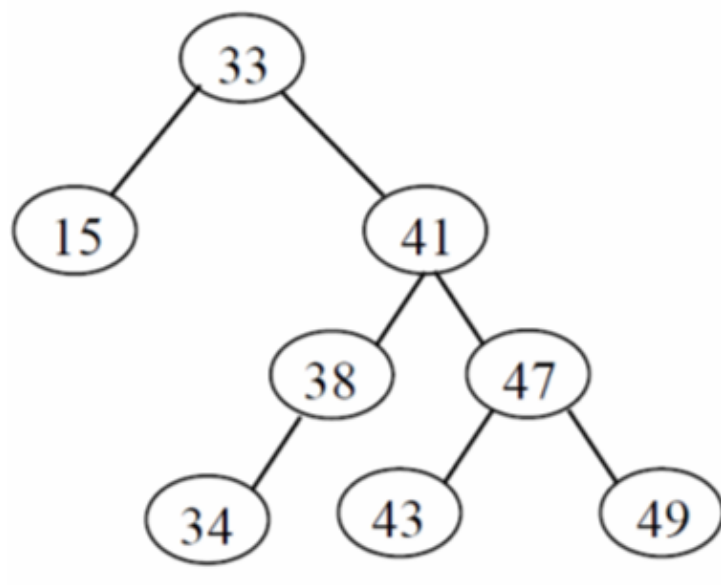
Comentário da Resposta**Resposta Resumida:**

Árvore de busca binária (BST): organiza os valores (menores à esquerda e maiores à direita), mas pode ficar desbalanceada.

Árvore AVL: é uma BST balanceada, onde a diferença de altura entre os lados é controlada (-1, 0 ou 1), mantendo a árvore mais eficiente para buscas.

Descrição

Descreva a ordem de visita para um percurso pós-ordem na árvore apresentada.

**Resposta Discursiva**

15, 34, 38, 43, 49, 47, 41, 33.

Nota Máxima

1,80

Nota

1,80

Comentário da Resposta

Descrição

Usando somente as funções do TAD Pilha(**ArrayStack**), implemente a função que determina se duas pilhas são iguais. Antes de retornar True(são iguais) ou False(são diferentes), as duas pilhas (pilha1 e pilha2) devem ter seu conteúdo restaurado. Usar como apoio duas pilhas auxiliares na sua solução. Duas pilhas são iguais se têm os mesmos elementos na mesma ordem ou são vazias.

```
def iguais_pilhas( pilha1, pilha2):
```

Classe ArrayStack

atributos, propriedades ou estados

aluno.uvix.br/AgendamentoProva/VisualizarCorrecaoAluno/17133020221?IdDisciplinaOferta=126836&token=xx0wzPPmGC8%3D

022 22:17

Portal do Aluno

self._data : armazena os dados da pilha

métodos

def __init__(self): construtor: cria e retorna uma pilha vazia

def __len__(self): retorna quantos elementos estão na pilha

def is_empty(self): retorna True se a pilha estiver vazia, senão False.

def push(self, e): adiciona o elemento e no topo da pilha

def top(self): retorna o elemento do topo da pilha, mas não o remove

def pop(self): remove e retorna o elemento do topo da pilha

Resposta Discursiva

Nota Máxima

1,45

Nota

0,00

Comentário da Resposta

não fez a questão.

Python

 Copiar

```
def iguais_pilhas(p1, p2):
    aux1, aux2 = ArrayStack(), ArrayStack()
    iguais = True

    while not p1.is_empty() and not p2.is_empty():
        e1, e2 = p1.pop(), p2.pop()
        aux1.push(e1)
        aux2.push(e2)
        if e1 != e2:
            iguais = False

    if not p1.is_empty() or not p2.is_empty():
        iguais = False

    while not aux1.is_empty():
        p1.push(aux1.pop())
    while not aux2.is_empty():
        p2.push(aux2.pop())

    return iguais
```

Descrição	A busca binária é uma técnica eficiente para buscar elementos em vetores ou listas. A única restrição desse método é que o vetor ou a lista estejam ordenados. Das seguintes sequências de acessos em um vetor, qual é válida em uma execução de busca binária? A 3, 5, 6 B 3, 7, 5 C 3, 1, 5 D 5, 3, 6 E 1, 5, 3	✓ ☐ ☐ ☐ ☐
Nota	0,65	

Descrição	O algoritmo de ordenação rápida utiliza um pivô para recursivamente ordenar o vetor. Em cada chamada da função de partição, os elementos menores que o pivô são colocados na esquerda e os maiores que o pivô na direita. Essa operação é repetida até que reste apenas um elemento. Dado o vetor [13, 77, 61, 93, 57] e o pivô 61(elemento central), qual das opções a seguir é válida após a primeira execução da função partição? A [13, 57, 61, 93, 77] B [57, 13, 61, 77, 93] C [93, 77, 61, 57, 13] D [13, 77, 57, 61, 93] E [13, 77, 61, 57, 93]	✗ ✓ ☐ ☐ ☐ ✓
Nota	0,00	

Descrição	Avaliando o TAD pilha é correto afirmar:	✓
	A um tipo de lista linear em que as operações de inserção e remoção são realizadas na extremidade denominada topo. B uma lista do tipo LIFO C uma lista do tipo FIFO D um tipo de lista linear em que as operações de inserção e remoção são realizadas aleatoriamente E um tipo de lista linear em que as operações de inserção são realizadas em uma extremidade e as operações de remoção são realizadas em outra extremidade	✓ ☐ ☐ ☐ ☐
Nota	0,65	

Descrição	Avalie as afirmativas sobre os TADs lista, pilha e fila:	✗
	I. As estruturas de dados pilhas, filas e listas armazenam coleções de itens. A característica que as distingue é a ordem em que podem ser retirados os itens dessas coleções em relação à ordem em que foram inseridos. II. Considere que os itens A, B, C, D, E foram inseridos nessa ordem em uma fila. Necessariamente, o primeiro elemento a ser removido dessa fila é o elemento A. III. Considere que os itens A, B, C, D, E foram inseridos nessa ordem em uma pilha. Necessariamente, o último elemento a ser removido dessa pilha é o elemento E. IV. Considere que os itens A, B, C, D, E foram inseridos nessa ordem em uma lista. Necessariamente, o primeiro elemento a ser removido dessa lista é o elemento A. Assinale a alternativa correta. A I e II somente B I e IV somente C III e IV somente D I, II e III somente E II, III e IV somente	✓ ☐ ☐ ✓ ☐
Nota	0,00	

Descrição

A busca sequencial é uma técnica simples e intuitiva para buscar elementos em vetores ou listas.



Tendo a informação de que os dados estão ordenados de forma crescente, qual otimização você pode aplicar na busca sequencial?

A Parar a execução e informar que o valor não existe, quando o valor atual for maior que o buscado



B Calcular a função *hash* e descobrir em qual posição o elemento deveria estar



C Descobrir se o elemento está na lista, apenas comparando o elemento do meio



D Parar a execução e informar que o valor não existe quando o valor atual for menor que o buscado



E Descobrir onde está o elemento buscado, testando apenas a primeira posição do vetor



Nota

0,00

Descrição

Em relação às estruturas de dados implementadas usando o tipo lista, analise as assertivas abaixo:

I - Uma implementação de fila por meio listas. Para enfileirar um item, basta mover usar o método pop; para desenfileirar um item, basta mover o método append.

II - Em uma lista duplamente encadeada, todas as inserções são realizadas em um extremo da lista, enquanto as exclusões e acessos são realizados no outro extremo da lista.

III - Filas são utilizadas quando se deseja processar itens de acordo com a ordem FIFO.

iluno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno/17798520231?idDisciplinaOferta=133490&token=vlok8KCumUg%3D

023, 15:04

Portal do Aluno

IV - Uma pilha é uma lista linear nas quais inserções, exclusões e acessos a itens ocorrem sempre em um dos extremos da lista.

Assinalando V, se verdadeiras, ou F, se falsas.

A ordem correta das afirmações de cima para baixo, é:



A

F – F – V – V



B

V – F – F – V



C

V – V – F – F



D

V – F – V – F



E

F – V – F – V



Descrição

No contexto das estruturas de dados, considere uma pilha (stack) onde as seguintes operações foram executadas. A pilha começa vazia.

Operações de LIFO:

top: retorna o primeiro elemento da Pilha, sem remover;

push: adiciona um elemento no topo da Pilha;

pop: remove o elemento do topo da Pilha e o retorna;

isEmpty: retorna true se a Pilha estiver vazia e false caso contrário.

Código a ser avaliado:

aluno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno/17798520231?idDisciplinaOferta=133490&token=vlok8KCumUg%3D

2023, 15:04

Portal do Aluno

```
P = Pilha()
P.push(12)
P.push(14)
P.pop()
P.push(20)
P.push(15)
if(P.top()%3 and P.top()%5):
    P.push(19)
else:
    P.pop()
```

Assinale a opção que indica o número de elementos e o valor do elemento localizado no topo da pilha, ao final das operações.



A
4 e 19



B
4 e 15



C
3 e 15



D
3 e 19



E
3 e 20

Descrição

aluno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno/17798520231?idDisciplinaOferta=133490&token=vlok8KCumUg%3D

023, 15:04

Portal do Aluno

Você foi contratado como consultor por uma empresa de desenvolvimento de sistemas. Um dos clientes solicitou o desenvolvimento de uma nova funcionalidade que permite ordenar os clientes por idade e renda. Essa funcionalidade será utilizada diversas vezes ao dia, o que indica que após uma ordenação, as outras só serão necessárias caso atualizações tenham sido feitas no banco de dados.

Quantas comparações serão feitas pelo algoritmo de ordenação por bolha, caso a entrada seja uma lista de 100 elementos já ordenados?



A

99



B

98



C

49



D

24



E

100



Nota

0,60

Dada a seguinte implementação do TAD Lista, determine o que será exibido na saída

10/04/2023, 15:04

Portal do Aluno

```
class ListNode:
    def __init__(self):
        self.data = None
        self.nextNode = None

def inicia_valores(node, elem, outroNode):
    node.data = elem
    node.nextNode = outroNode

if __name__ == '__main__':
    n = [ListNode(), ListNode()]
    inicia_valores(n[0], 'a', n[1])
    inicia_valores(n[1], 'b', n[0])
    print("1:", n[0].data)
    print("2:", n[0].nextNode.data)
    print("3:", n[1].nextNode.data)
    print("4:", n[1].nextNode.nextNode.data)
```

A

1: a

2: b

3: a

4: b

✓

B

1: a

2: a

3: b

4: b

✓

<https://aluno.uv.vb/AgendamentoProva/VisualizarCorrecaoAluno/1779852023171x5DisciplinaOferta=133490&token=viok8KCumUp%3D>

10/04/2023, 15:04

Portal do Aluno

C

1: b

2: b

3: a

4: b

□

D

1: a

2: b

3: b

4: a

□

E

1: a

2: b

3: a

4: a

□

Nota

0,00

Descrição

As Filas ou FIFO's (First In – First Out) e as Pilhas ou LIFO's (Last In – First Out) são estruturas que permitem a manipulação de dados de maneira transparente. Considere uma FIFO F com os seguintes elementos F=[10,20,30,40] (esquerda representa o início e direita o final) e uma LIFO L vazia, L=[]

aluno.uv.br/AgendamentoProva/VisualizarCorrecaoAluno/177985202317?disciplinaOferta=133490&token=vtok8KCumUg%3D

2023, 15:04

Portal do Aluno

(esquerda representa a base e direita o topo).

L.push(F.first())

F.dequeue()

L.push(F.dequeue())

L.push(F.first())

F.dequeue()

L.push(F.first())

F.dequeue()

F.enqueue(L.top())

L.pop()

If(!F.is_empty() and F.first() % 3 < 2):

L.push(L.top() + 10)

Assinale a alternativa que represente os estados da FIFO F e da LIFO L após a execução da sequência de operações descritas acima:



A

F=[40], L=[10,20,30,40]



B

F=[40], L=[10,20,30,50]



C

F=[], L=[40,30,20,10]



D

F=[40], L=[40,30,20,10]



E

F=[], L=[10,20,30,40]



Nota

0,60

Dada a implementação do TAD Pilha em Python (class Stack), qual implementação da função verifica_balanceamento_parenteses está correta ao ser usada para avaliar se uma sequência de parênteses (string de parênteses somente) está bem formada (para cada abre parênteses tem um fecha parênteses) ou não:

```
class Stack:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def peek(self):
        return self.items[len(self.items)-1]

    def size(self):
        return len(self.items)
```



A

```
def verifica_balanceamento_parenteses(string):
    pilha = Stack()
    esta_balanceado = True
    posicao = 0
    while posicao < len(string) and esta_balanceado:
        caracter_lido = string[posicao]
        if caracter_lido == "(":
            pilha.push(caracter_lido)
        else:
            if pilha.isEmpty():
                esta_balanceado = False
            else:
                pilha.pop()
        posicao = posicao + 1
    if esta_balanceado and pilha.isEmpty():
        return True
    else:
        return False
```



B

```
def verifica_balanceamento_parenteses(string):
    pilha = Stack()
    esta_balanceado = True
    posicao = 0
    while posicao < len(string) and not esta_balanceado:
        caracter_lido = string[posicao]
        if caracter_lido == "(":
            pilha.push(caracter_lido)
        else:
            if pilha.isEmpty():
                esta_balanceado = False
            else:
                pilha.pop()
        posicao = posicao + 1
    if esta_balanceado and pilha.isEmpty():
        return True
    else:
        return False
```



C

```
def verifica_balanceamento_parenteses(string):
    pilha = Stack()
    esta_balanceado = False
    posicao = 0
    while posicao < len(string) and not esta_balanceado:
        caracter_lido = string[posicao]
        if caracter_lido == "(":
            pilha.push(caracter_lido)
        else:
            if pilha.isEmpty():
                esta_balanceado = False
            else:
                pilha.pop()
        posicao = posicao + 1
    if esta_balanceado and pilha.isEmpty():
        return True
    else:
        return False
```



D

```
def verifica_balanceamento_parenteses(string):
    pilha = Stack()
    esta_balanceado = False
    posicao = 0
    while posicao < len(string) and not esta_balanceado:
        caracter_lido = string[posicao]
        if caracter_lido == "(":
            pilha.push(caracter_lido)
        else:
            if pilha.isEmpty():
                esta_balanceado = False
            else:
                pilha.pop()
        posicao = posicao + 1
    if esta_balanceado and pilha.isEmpty():
        return True
    else:
        return False
```



E

```
def verifica_balanceamento_parenteses(string):
    pilha = Stack()
    esta_balanceado = False
    posicao = 0
    while posicao < len(string) and not esta_balanceado:
        caracter_lido = string[posicao]
        if caracter_lido == "(":
            pilha.push(caracter_lido)
        else:
            if pilha.isEmpty():
                esta_balanceado = True
            else:
                pilha.pop()
        posicao = posicao + 1
    if esta_balanceado and pilha.isEmpty():
        return False
    else:
        return True
```



Nota
0,00

Descrição

As Filas ou FIFO's (First In – First Out) e as Pilhas ou LIFO's (Last In – First Out) são estruturas que permitem a manipulação de dados de maneira transparente. Considere uma FIFO F com os seguintes elementos F=[80,3,5,9,10,13] (esquerda representa o início e direita o final) e uma LIFO L vazia, L=[] (esquerda representa a base e direita o topo). Com base na sequência de instruções a seguir, assinale a alternativa que mostre o estado de F e L após a execução da seguinte sequência de operações:

Operações de FIFO:

first: retorna o primeiro elemento da Fila, sem remover;
enqueue: adiciona um elemento no final da Fila;
dequeue: remove o primeiro elemento da Fila e o retorna;
isEmpty: retorna true se a Fila estiver vazia e false caso contrário.

Operações de LIFO:

top: retorna o primeiro elemento da Pilha, sem remover;
push: adiciona um elemento no topo da Pilha;
pop: remove o elemento do topo da Pilha e o retorna;
isEmpty: retorna true se a Pilha estiver vazia e false caso contrário.

Código a ser avaliado:

```
num1 = 2
num2 = 3
while valendo:
    if(F.first()%num1 != 0 and F.first()%num2 !=0):
        L.push(F.dequeue())
        valendo = F.isEmpty()
    elif (not(not L.isEmpty())):
        L.push(F.first())
        F.dequeue()
        F.enqueue(L.top())
num2 = 2
```



A

F=[9,10,13,80,3] e L=[80,3]



B

F=[9,10,13,80,3] e L=[80]



C

F=[9,10,13,80,3] e L=[3]



D

F=[3,9,10,13,80,3] e L=[80,3]



E

F=[3,9,10,13,80,3] e L=[3]

Descrição

Assinale alternativa correta sobre o TAD Lista Simplesmente Encadeada:

**A**

A implementação de lista por meio de apontadores/referências permite utilizar posições não contíguas de memória, de modo a se poder inserir e retirar elementos sem que haja necessidade de deslocar os itens seguintes da lista.

**B**

Só é possível inserir elementos no final da estrutura, ou seja, no ponteiro para o último elemento

☐**C**

Utiliza o mecanismo FIFO, ou seja, o primeiro elemento a entrar é o primeiro elemento a sair da estrutura

☐**D**

Não utiliza o mecanismo LIFO, ou seja, o último elemento a ser inserido não pode ser o primeiro a ser removido

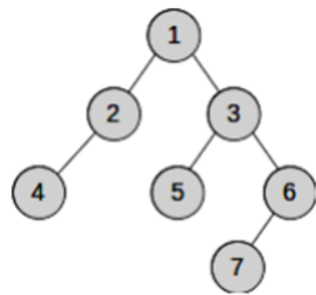
☐**E**

Não é possível implementá-las com arrays

☐**Nota**

0,60

(AOCP - 2021) Dada a árvore a seguir, assinale a alternativa que apresenta o passeio em ordem para essa árvore.



- A 1 2 4 3 5 6 7
- B 1 2 3 4 5 6 7
- C 4 2 1 7 5 6 3
- D 4 2 1 5 3 7 6
- E 7 4 5 6 2 3 1

☐

☐

☐

☒

☐

0,60

(IF-SP - 2019). Abaixo tem-se uma tabela que ilustra o conjunto de nós de uma lista duplamente encadeada, contendo o total de 5 nós.



elemento	anterior	proximo	conteúdo
1	3	4	?
2		5	?
3	5	1	?
4	1		?
5	2	3	?

Ao imprimir a estrutura na ordem correta, o conteúdo apresentado será I – F – S – P – 2019, dessa forma, assinale a alternativa que contém os dados que preenchem, corretamente, a coluna “conteúdo”, de cima para baixo.

- A P – I – S – 2019 – F
- B 2019 – P – S – F – I
- C S – P – I – F – 2019
- D S – I – F – 2019 – P
- E Nenhuma das anteriores

☒

☐

☐

☐

☐

0,60

Descrição

Dada a sequência numérica (25, 18, 12, 30, 8, 16, 22, 10, 5, 20), qual é a sequência parcialmente ordenada após a segunda passagem do Bubble Sort?

**A**

18, 12, 25, 8, 16, 22, 10, 5, 20, 30

☐**B**

25, 12, 18, 8, 16, 10, 5, 20, 22, 30

☐**C**

12, 18, 8, 16, 22, 10, 5, 20, 25, 30

**D**

12, 8, 16, 18, 10, 5, 20, 22, 25, 30

**E**

25, 18, 12, 30, 8, 16, 22, 10, 5, 20

☐**Nota**

0,00

Descrição

aluno.uvv.br/AgendamentoProva/VisualizarCorrecaoAluno/18742520241?idDisciplinaOferta=142930&token=aMh%2Fe61CkTg%3D

Página

o Aluno

12/04/20

Dada a sequência numérica (6, 10, 4, 8, 2, 12, 14, 16, 18, 20), qual é a sequência parcialmente ordenada após a terceira passagem do Bubble Sort?



A

6, 10, 4, 8, 2, 12, 14, 16, 18, 20

☐

B

6, 4, 8, 2, 10, 12, 14, 16, 18, 20

☐

C

4, 6, 2, 8, 10, 12, 14, 16, 18, 20

☐

D

4, 2, 6, 8, 10, 12, 14, 16, 18, 20



E

6, 10, 4, 8, 2, 12, 14, 16, 18, 20

☐

Nota

0,60

Descrição

(ENADE – 2011) No desenvolvimento de um software que analisa bases de DNA, representadas pelas letras A, C, G, T, utilizou-se as estruturas de dados: pilha e fila. Considere que, se uma sequência representa uma pilha, o topo é o elemento mais à esquerda; e se uma sequência representa uma fila, a

sua frente é o elemento mais à esquerda. Analise o seguinte cenário: “a sequência inicial ficou armazenada na primeira estrutura de dados na seguinte ordem: (A,G,T,C,A,G,T,T). Cada elemento foi retirado da primeira estrutura de dados e inserido na segunda estrutura de dados, e a sequência ficou armazenada na seguinte ordem: (T,T,G,A,C,T,G,A). Finalmente, cada elemento foi retirado da segunda estrutura de dados e inserido na terceira estrutura de dados e a sequência ficou armazenada na seguinte ordem: (T,T,G,A,C,T,G,A)”. Qual a única sequência de estruturas de dados apresentadas a seguir pode ter sido usada no cenário descrito acima?



A

Fila – Pilha – Fila



B

Fila – Fila – Pilha



C

Fila – Pilha – Pilha



D

Pilha – Fila – Pilha



E

Pilha – Pilha – Pilha



Descrição

Qual é a principal característica de uma árvore AVL?

**A**

Possui um fator de balanceamento igual a 1

☐**B**

Mantém-se balanceada após inserções e remoções

**C**

É uma árvore binária sem ordenação específica

☐**D**

Tem um tempo de busca pior que de uma lista encadeada simples

☐**E**

Nenhuma das anteriores

☐**Nota**

0,60

Descrição

Em uma árvore AVL, como é calculado o fator de balanceamento de um nó?

**A**

Grau da subárvore esquerda menos grau da subárvore direita

☐**B**

Altura da subárvore esquerda menos altura da subárvore direita

**C**

Grau da subárvore esquerda mais grau da subárvore direita

☐**D**

Altura da subárvore esquerda mais altura da subárvore direita

☐**E**

Nenhuma das anteriores

☐**Nota**

0,60

Uma pilha é uma estrutura de dados que armazena uma coleção de itens de dados relacionados e que garante o seguinte funcionamento: o último elemento a ser inserido é o primeiro a ser removido. É comum na literatura utilizar os nomes push e pop para as operações de inserção e remoção de um elemento em uma pilha, respectivamente. O seguinte trecho de código em Linguagem Python define uma estrutura de dados pilha utilizando um vetor de inteiros, bem como algumas funções para sua manipulação.

```
class ArrayStack:
    def __init__(self):
        self._data = [None] * 100
        self.topo = -1 # pilha vazia

    def is_empty(self):
        return self.topo == -1

    def push(self, e):
        if(self.topo >= 99):
            return
        self.topo += 1
        self._data[self.topo] = e

    def top(self):
        if self.is_empty():
            print('pilha vazia')
            return None
        else:
            return self._data[self.topo]

    def pop(self):
        if self.is_empty():
            print('pilha vazia')
            return None
        else:
            elem = self._data[self.topo]
            self.topo -= 1
            return elem

if __name__ == '__main__':
    S = ArrayStack()
    S.push(2)
    S.push(3)
    S.push(4)
    S.pop()
    S.push(2)
    elem = S.pop() + S.pop()
    S.push(elem)
    elem += S.pop()
    print(elem)
```

- A esse respeito, avalie as afirmações a seguir.
- I. O valor no topo da pilha S é 2
 - II. O valor exibido pelo programa seria o mesmo caso a instrução `elem += S.pop()` fosse trocada por `elem+=elem;`
 - III. O valor no topo da pilha S é 5

É correto o que se afirma em:

- A**
a) I e II apenas.

- B**
a) I, apenas.

- C**
a) III, apenas.

- D**
a) II e III apenas.

- E**
a) I, II e III apenas.

Nota
0,60

Descrição

Assinale a alternativa que implementa corretamente a função que verifica se um dado elemento esta presente na fila, sem destruir a fila original.



aluno.uvvi.br/Agendamento/Prova/Visualizar/CorrecaoAluno?parametros=B1gQzJRmasgaMAISsgajhCzo17dFX0n0w1ocbtLS8hVA1b32uq6A.

02/04, 09:11

Portal do Aluno

A

```
def existeElemento(fila, elem):
    if( fila.is_empty()):
        return False
    else:
        copia = copiarFila(fila)
        while(copia):
            if(elem == copia.dequeue()):
                copia = None
                return True
        copia = None
        return False
```



B

```
def existeElemento(fila, elem):
    if( fila.is_empty()):
        return False
    else:
        copia = copiarFila(fila)
        while(copia):
            if(elem == copia.dequeue()):
                copia = None
                return False
        copia = None
        return True
```



C

04, 09:11

Portal do Aluno

```
def existeElemento(fila, elem):
    if( fila.is_empty()):
        return False
    else:
        copia = copiarFila(fila)
        while(copia):
            if(elem != copia.dequeue()):
                copia = None
                return True
        copia = None
        return False
```



D

```
def existeElemento(fila, elem):
    if( fila.is_empty()):
        return True
    else:
        copia = copiarFila(fila)
        while(copia):
            if(elem != copia.dequeue()):
                copia = None
                return True
        copia = None
        return False
```



O algoritmo a seguir recebe um vetor v de números inteiros e rearranja esse vetor de tal forma que seus elementos, ao final, estejam ordenados de forma crescente.



```
01 def ordena(v):
02     n = len(v)
03     for i in range(1, n):
04         chave = v[i]
05         j = i - 1
06         while j >= 0 and v[j] < chave:
07             v[j + 1] = v[j]
08             j -= 1
09         v[j + 1] = chave
```

Considerando que nesse algoritmo há erros de lógica que devem ser corrigidos para que os elementos sejam ordenados de forma crescente, assinale a opção correta no que se refere às correções adequadas.

- A** A linha 03 deve ser corrigida da seguinte forma: `for i in range(1, n-1):` e a linha 09, do seguinte modo: `v[j - 1] = chave` ☐
- B** A linha 06 deve ser corrigida da seguinte forma: `while j >= 0 and v[j] > chave` e a linha 07, do seguinte modo: `v[j + 1] = v[j]` ☒
- C** A linha 05 deve ser corrigida da seguinte forma: `j = i + 1` e a linha 06, do seguinte modo: `while j >= 0 and v[j] > chave` ☐
- D** A linha 07 deve ser corrigida da seguinte forma: `v[j + 1] = v[j]` e a linha 09, do seguinte modo: `v[j - 1] = chave` ☐
- E** A linha 04 deve ser corrigida da seguinte forma: `for i in range(1, n-1):` e a linha 05, do seguinte modo: `j = i + 1` ☐

0,60

Descrição Em relação às estruturas de dados do tipo lista, analise as assertivas abaixo, assinalando V, se verdadeiras, ou F, se falsas.



() Uma implementação de fila por meio de arranjos é circular e delimitada pelos apontadores Frente e Trás. Para enfileirar um item, basta mover o apontador Trás uma posição no sentido horário; para desenfileirar um item, basta mover o apontador Frente no sentido horário.

() Em uma lista duplamente encadeada, todas as inserções são realizadas em um extremo da lista, enquanto as exclusões e acessos são realizados no outro extremo da lista.

() Filas são utilizadas quando se deseja processar itens de acordo com a ordem FIFO.

() Uma pilha é uma lista linear nas quais inserções, exclusões e acessos a itens ocorrem sempre em um dos extremos da lista.

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

- A** F - F - V - V
- B** V - F - F - V
- C** V - V - F - F
- D** V - F - V - F
- E** F - V - F - V



Nota

0,60

Descrição

Considerando a Fila (ArrayQueue) como um Vetor Circular, conforme o construtor descrito abaixo, determine qual a implementação correta da função **def enqueue(elem)**:

class ArrayQueue:

DEFAULT_CAPACITY = 100

```
def __init__(self):
    self._data = [None] * ArrayQueue.DEFAULT_CAPACITY # vetor circular
    self._size = 0 # quantidade de elementos adicionados
    self._first = -1 # posição do primeiro elemento
    self._last = -1 # posição do último elemento
```

Qual a implementação correta:



A

```
def enqueue(self, e):
    if self.is_empty():
        self._data[self._last + 1] = e
        self._size += 1
        self._first = self._last = self._first + 1
    else if not ((self._last + 1) % ArrayQueue.DEFAULT_CAPACITY == self._first):
        self._last = (self._last + 1) % ArrayQueue.DEFAULT_CAPACITY - 1
        self._data[self._last] = e

    self._size += 1
```

☐

B

```
def enqueue(self, e):
    if self.is_empty():
        self._data[self._first + 1] = e
        self._first = self._last = self._first + 1
    else if not ((self._last + 1) % ArrayQueue.DEFAULT_CAPACITY == self._first):
        self._last = (self._last) % ArrayQueue.DEFAULT_CAPACITY
        self._data[self._last] = e
```

self._size += 1

☐

C

```
def enqueue(self, e):
    if self.is_empty():
        self._data[self._first] = e
        self._size += 1
        self._first = self._last = 0
    else if not ((self._last + 1) % ArrayQueue.DEFAULT_CAPACITY == self._first):
        self._last = (self._last + 1) % ArrayQueue.DEFAULT_CAPACITY
        self._data[self._last] = e

    self._size += 1
```

☐

D

```
def enqueue(self, e):
    if self.is_empty():
        self._data[self._last + 1] = e
        self._size += 1
        self._first = self._last = self._first + 1
    else if ((self._last + 1) % ArrayQueue.DEFAULT_CAPACITY == self._first):
        self._last = (self._last + 1) % ArrayQueue.DEFAULT_CAPACITY
        self._data[self._last] = e

    self._size += 1
```

☐

E

```
def enqueue(self, e):
    if self.is_empty():
        self._data[self._last + 1] = e
        self._size += 1
        self._first = self._last = self._first + 1
    else if not ((self._last + 1) % ArrayQueue.DEFAULT_CAPACITY == self._first):
        self._last = (self._last + 1) % ArrayQueue.DEFAULT_CAPACITY
        self._data[self._last] = e

    self._size += 1
```



Nota

0,60

Descrição

Em relação às estruturas de dados do tipo lista, analise as assertivas abaixo, assinalando V, se verdadeiras, ou F, se falsas.

() Uma implementação de fila por meio de arranjos é circular e delimitada pelos apontadores Frente e Trás. Para enfileirar um item, basta mover o apontador Trás uma posição no sentido horário; para desenfileirar um item, basta mover o apontador Frente no sentido horário.

() Em uma lista duplamente encadeada, todas as inserções são realizadas em um extremo da lista, enquanto as exclusões e acessos são realizados no outro extremo da lista.

() Filas são utilizadas quando se deseja processar itens de acordo com a ordem FIFO.

() Uma pilha é uma lista linear nas quais inserções, exclusões e acessos a itens ocorrem sempre em um dos extremos da lista.

A ordem correta de preenchimento dos parênteses, de cima para baixo, é:

A

F – F – V – V.

B

V – F – F – V.

C

V – V – F – F.

D

V – F – V – F.

E

F – V – F – V.

Nota

0,00



Descrição

As listas encadeadas são estruturas versáteis que permitem que uma quantidade indeterminada de dados seja armazenada em tempo de execução na memória primária de um computador.

Com base no texto acima, avalie seguintes asserções e a relação proposta entre elas.

- I. A única forma de se navegar em uma lista encadeada é por meio dos endereços dos Nós, que geralmente são salvos em campos específicos da estrutura do No ou struct ListaNo.

PORQUE

- II. De forma similar aos vetores, uma lista encadeada é contígua na memória.

A respeito dessas asserções, assinale a alternativa correta.

**A**

A asserção I é uma proposição verdadeira, e a II é uma proposição falsa

**B**

- a) As asserções I e II são proposições verdadeiras, e a II é justificativa da I.

**C**

- a) As asserções I e II são proposições verdadeiras, mas a II não é justificativa da I.

**D**

A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.

**E**

- a) As asserções I e II são proposições falsas.



Descrição

Árvores binárias possibilitam maior organização dos elementos adicionados, o que facilita separar os elementos a cada nível de crescimento da árvore.

Dada a seguinte lista de caracteres [P, R, O, G, R, A, M, A, D, O, R, U, M], como ficaria seu resultado final em uma leitura pré-ordem, caso seus valores fossem adicionados sequencialmente, do primeiro ao último elemento, em uma árvore binária? Para montar uma árvore binária, os elementos menores ou iguais ao nó mais abaixo entrarão à esquerda; caso contrário, ficarão à direita.



A

POGAADMMORRRU



B

AADGMMOOPRRRU



C

ADAMOMGORRURP



D

MOOPRRRRUGORR



E

ADAMOMGORURRP



Nota

0,00

Descrição

As Filas ou FIFO's (First In – First Out) e as Pilhas ou LIFO's (Last In – First Out) são estruturas que permitem a manipulação de dados de maneira transparente. Considere uma FIFO F com os seguintes elementos F=[80,3,5,9,10,13] (esquerda representa o início e direita o final) e uma LIFO L vazia, L=[] (esquerda representa a base e direita o topo). Com base na sequência de instruções a seguir, assinale a alternativa que mostre o estado de F e L após a execução da seguinte sequência de operações:

Operações de FIFO:

first: retorna o primeiro elemento da Fila, sem remover;

enqueue: adiciona um elemento no final da Fila;

dequeue: remove o primeiro elemento da Fila e o retorna;

isEmpty: retorna true se a Fila estiver vazia e false caso contrário.

Operações de LIFO:

top: retorna o primeiro elemento da Pilha, sem remover;

push: adiciona um elemento no topo da Pilha;

pop: remove o elemento do topo da Pilha e o retorna;

isEmpty: retorna true se a Pilha estiver vazia e false caso contrário.

Código a ser avaliado:

```
num1 = 2
num2 = 3
while valendo:
    if(F.first()%num1 != 0 and F.first()%num2 !=0):
        L.push(F.dequeue())
        valendo = F.isEmpty()
    elif (not(not L.isEmpty())):
        L.push(F.first())
        F.dequeue()
        F.enqueue(L.top())
num2 = 2
```



A

F=[9,10,13,80,3] e L=[80,3]



B

F=[9,10,13,80,3] e L=[80]



C

F=[9,10,13,80,3] e L=[3]



D

F=[3,9,10,13,80,3] e L=[80,3]



E

F=[3,9,10,13,80,3] e L=[3]

Descrição

Sejam as operações de Fila abaixo:

Operações de FIFO:

Fila: construtor;

first: retorna o primeiro elemento da Fila, sem remover;

enqueue: adiciona um elemento no final da Fila;

dequeue: remove o primeiro elemento da Fila e o retorna;

isEmpty: retorna true se a Fila estiver vazia e false caso contrário.

Qual código imprime o conteúdo de uma Fila e depois restaura a Fila original?

A

```
def printFila(q):  
    copia = Fila()  
    print('[', end="")  
    while(not q.is_empty()):  
        print(q.first(), end=" ")  
        copia.enqueue(q.dequeue())  
    while(not copia.is_empty()):  
        q.enqueue(copia.dequeue())  
    copia = None  
    print(']\n')
```



B

```
def printFila(q):  
    print('[', end="")  
    while(not q.is_empty()):  
        print(q.first(), end=" ")  
        copia.enqueue(q.dequeue())  
    while(not copia.is_empty()):  
        q.enqueue(copia.dequeue())  
    copia = None  
    print(']\n')
```



C

```
def printFila(q):  
    copia = Fila()  
    print('[', end="")  
    while(not q.is_empty()):  
        print(q.first(), end=" ")  
    while(not copia.is_empty()):  
        q.enqueue(copia.dequeue())  
    copia = None  
    print(']\n')
```



D

```
def printFila(q):  
    copia = Fila()  
    print('[', end="")  
    while(not q.is_empty()):  
        print(q.first(), end=" ")  
        copia.enqueue(q.dequeue())  
    while(not copia.is_empty()):  
        q.enqueue(copia.enqueue())  
    copia = None  
    print(']\n')
```



